DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN
ESCUELA DE INGENIERÍA
PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE

IIC2343 - Arquitectura de Computadores (II/2026)

Documento de preparación para la Actividad 1

Descripción

El presente documento tiene por objeto preparar al estudiante para la primera actividad de laboratorio. Se recomienda llegar habiendo practicado los bloques básicos (*Half-Adder* y *Full-Adder*) y, en particular, habiendo ensayado su conexión a la Basys3 mediante el archivo `Basys3.vhd` (interruptores como entrada y LEDs como salida).

Contenidos a tener en cuenta

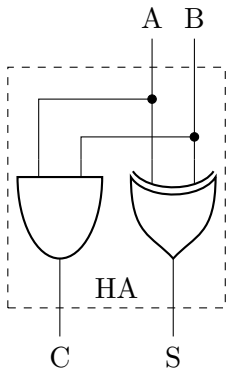
La actividad se centra en la descripción de circuitos combinacionales en VHDL mediante **entidades**, **componentes** e **instancias**. Los pilares a practicar son:

- **Half-Adder:** suma de dos bits; salidas de suma (`S`) y acarreo (`C`).
- **Full-Adder:** suma de dos bits con acarreo de entrada (`Cin`); se construye reutilizando el HA.
- **Interfaz física:** en `Basys3.vhd` se mapean los puertos del componente a `sw` y `led`.
- **Composición:** uso de instancias de un bloque dentro de otro (HA dentro de FA; FA dentro de un sumador más ancho).

1. Half-Adder

El *Half-Adder* es el bloque más básico de un sumador. Recibe dos bits y produce:

- `S`: bit de **suma**.
- `C`: bit de **acarreo**.



(a) Diagrama del Half-Adder

A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

(b) Tabla de verdad

Figura 1: Half-Adder: diagrama y comportamiento esperado.

Interpretación de la tabla

- Si ambos bits son 0, no hay resultado que reportar: `S`=0 y `C`=0.
- Si exactamente uno de los bits es 1, la suma es 1 y no hay acarreo.
- Si ambos son 1, en un solo bit no cabe el valor 2: la suma queda en 0 y el acarreo vale 1 (equivalente a escribir $1 + 1 = 10$ en binario).

De la tabla a VHDL

La columna `S` vale 1 cuando `A` y `B` son distintos, lo que corresponde a un `XOR`. La columna `C` vale 1 solo cuando ambos son 1, lo que corresponde a un `AND`. El comportamiento del componente queda descrito así:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;


entity HA is
    Port ( a : in std_logic;
           b : in std_logic;
           s : out std_logic;
           c : out std_logic);
end HA;

architecture Behavioral of HA is

begin

s <= a xor b;
c <= a and b;

end Behavioral;
```

Conexión en `Basys3.vhd`

Es necesario **instanciarlo** en `Basys3.vhd` y conectar sus puertos a interruptores y LEDs. Un esquema típico es:

1. Declarar el `component HA` (mismos puertos que la entidad).
2. Declarar señales internas para entradas y salidas.
3. Asignar bits de `sw` a las entradas.
4. Instanciar el HA con `port map`.
5. Asignar las salidas a bits de `led`.

Ejemplo de `Basys3.vhd` para probar el HA (dos interruptores y dos LEDs):

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Basys3 is
    Port (
        sw  : in  std_logic_vector (15 downto 0);
        led : out std_logic_vector (15 downto 0)
    );
end Basys3;
architecture Behavioral of Basys3 is
    component HA
        Port (
            a : in  std_logic;
            b : in  std_logic;
            s : out std_logic;
            c : out std_logic
        );
    end component;
    signal a_i : std_logic;
    signal b_i : std_logic;
    signal s_i : std_logic;
    signal c_i : std_logic;
begin
    -- Mapeo desde interruptores hacia las entradas del HA
    a_i <= sw(0);
    b_i <= sw(1);
    -- Instancia del Half-Adder
    inst_HA: HA port map(
        a => a_i,
        b => b_i,
        s => s_i,
        c => c_i
    );
    -- Mapeo desde las salidas del HA hacia los LEDs
    led(0) <= s_i;
    led(1) <= c_i;
    -- El resto de LEDs se deja en '0' para evitar valores indefinidos
    led(15 downto 2) <= (others => '0');
end Behavioral;
```

Con esta conexión, la verificación consiste en recorrer los cuatro casos de la tabla moviendo únicamente `sw(0)` y `sw(1)`, y comprobar que `led(0)` y `led(1)` coinciden con `S` y `C`.

2. Full-Adder

El *Full-Adder* suma tres bits de entrada: `A`, `B` y un acarreo entrante `Cin`. Entrega `S` y `Cout`.

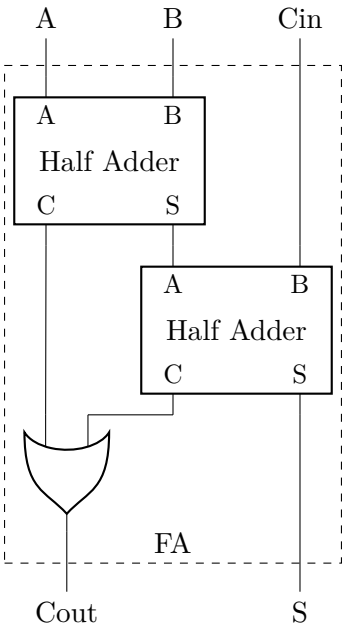


Figura 2: Full-Adder construido a partir de Half-Adder. Las conexiones entre instancias se deducen del diagrama.

`A`	`B`	`Cin`	`S`	`Cout`
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

El FA reutiliza el HA mediante **dos instancias** y combina los acarreos parciales. En VHDL ello implica declarar el `component HA`, definir señales internas e instanciar. A modo de estructura (sin completar el `port map`):

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Esqueleto del Full-Adder: se declara el componente HA,
-- se definen senales internas y se instancian dos HA.
-- Las conexiones exactas deben deducirse del diagrama del FA.
entity FA is
    Port (
        a  : in  std_logic;
        b  : in  std_logic;
        ci : in  std_logic;
        s  : out std_logic;
        c  : out std_logic
    );
end FA;

architecture Behavioral of FA is

    component HA
        Port (
            a : in  std_logic;
            b : in  std_logic;
            s : out std_logic;
            c : out std_logic
        );
    end component;

    signal s_parcial : std_logic;
    signal c_parcial1 : std_logic;
    signal c_parcial2 : std_logic;

begin

    -- Primera instancia de HA (completar port map segun el diagrama)
    -- inst_HA1: HA port map( ... );

    -- Segunda instancia de HA (completar port map segun el diagrama)
    -- inst_HA2: HA port map( ... );

    -- El acarreo de salida combina los acarreos parciales
    -- c <= ... ;

end Behavioral;
```

Conexión en `Basys3.vhd`

Una vez descrito el FA, el archivo `Basys3.vhd` deja de instanciar el HA y pasa a instanciar el FA. El patrón es el mismo: declaración del componente, señales, mapeo desde `sw`, `port map` y mapeo hacia `led`.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Ejemplo de Basys3 para probar el Full-Adder en la placa.
-- Entradas: sw(0)=A, sw(1)=B, sw(2)=Cin
-- Salidas:  led(0)=S, led(1)=Cout
entity Basys3 is
    Port (
        sw  : in  std_logic_vector (15 downto 0);
        led : out std_logic_vector (15 downto 0)
    );
end Basys3;

architecture Behavioral of Basys3 is

    component FA
        Port (
            a  : in  std_logic;
            b  : in  std_logic;
            ci : in  std_logic;
            s  : out std_logic;
            c  : out std_logic
        );
    end component;

    signal a_i   : std_logic;
    signal b_i   : std_logic;
    signal cin_i : std_logic;
    signal s_i   : std_logic;
    signal cout_i: std_logic;

begin

    -- Mapeo desde interruptores hacia las entradas del FA
    a_i   <= sw(0);
    b_i   <= sw(1);
    cin_i <= sw(2);

    -- Instancia del Full-Adder
    inst_FA: FA port map(
        a  => a_i,
        b  => b_i,
        ci => cin_i,
        s  => s_i,
        c  => cout_i
    );

    -- Mapeo desde las salidas del FA hacia los LEDs
    led(0) <= s_i;
    led(1) <= cout_i;
    led(15 downto 2) <= (others => '0');

end Behavioral;
```

Para la verificación en placa, corresponde contrastar al menos los casos `Cin`=1 con `A`=`B`=0; `A`=`B`=1 con `Cin`=0; y `A`=`B`=`Cin`=1. Si el HA era correcto y el FA falla, el problema suele estar en las conexiones internas del FA (`port map` y señales), no en el mapeo de `Basys3.vhd`.

Recordatorio de VHDL

- VHDL describe hardware concurrente.
- `'0'`/`'1'` corresponden a `std_logic`; `"01"` corresponde a un `std_logic_vector`.
- Operadores frecuentes: `and`, `or`, `xor`, `not`.
- Acceso a un bit: `vec(0)`; a un rango: `vec(1 downto 0)`.
- Reutilización de un diseño: `component` + `signal` + `port map`.
- En `Basys3.vhd`, los puertos `sw` y `led` no se modifican; solo cambia lo que se instancia y cómo se conecta.